

Animal behavior analysis in python

Day 2 Deeplabcut workflow

Miao Wang wang.miao@campus.lmu.de

Resources

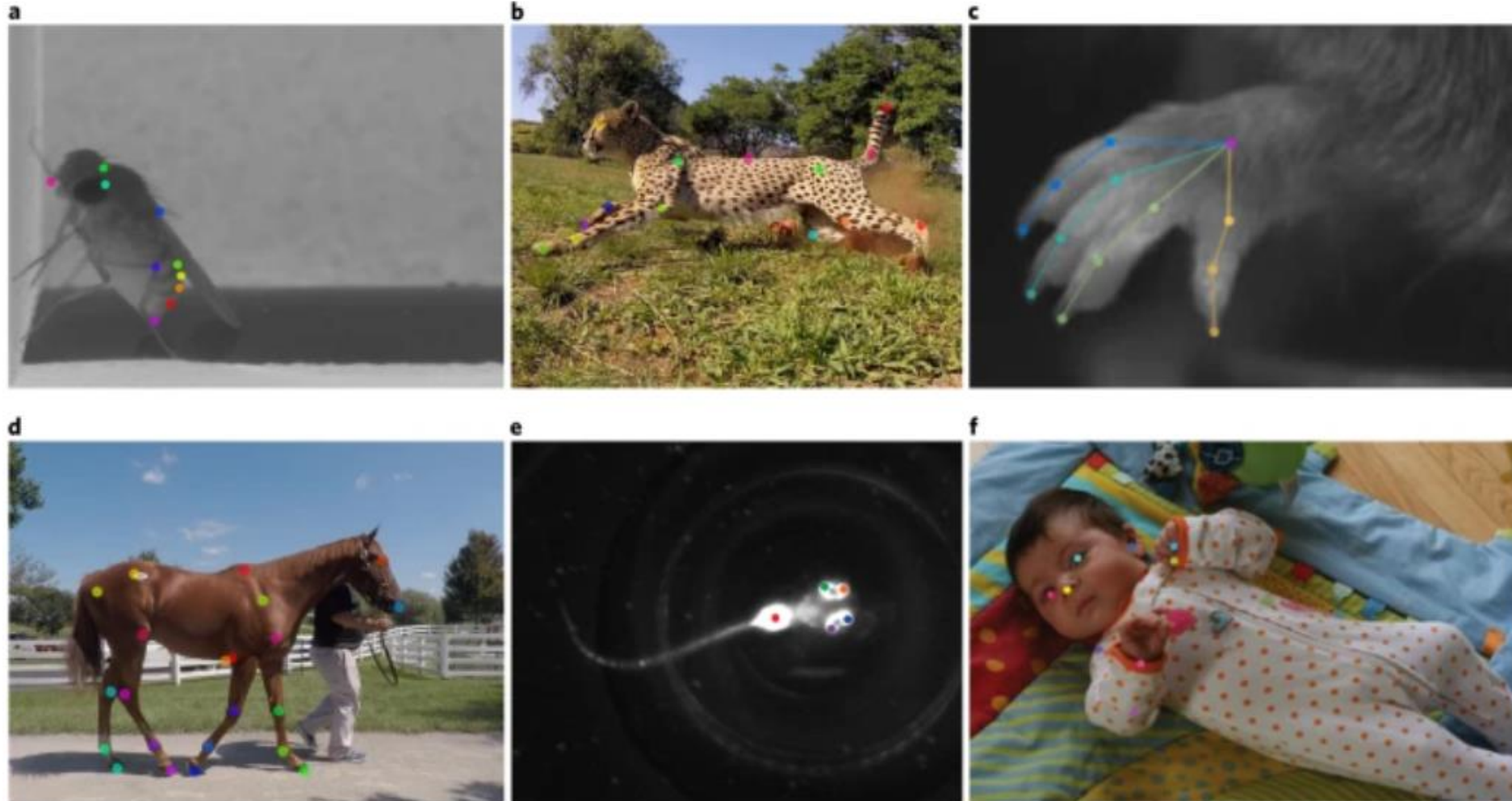
Preprint | Published: 21 June 2019

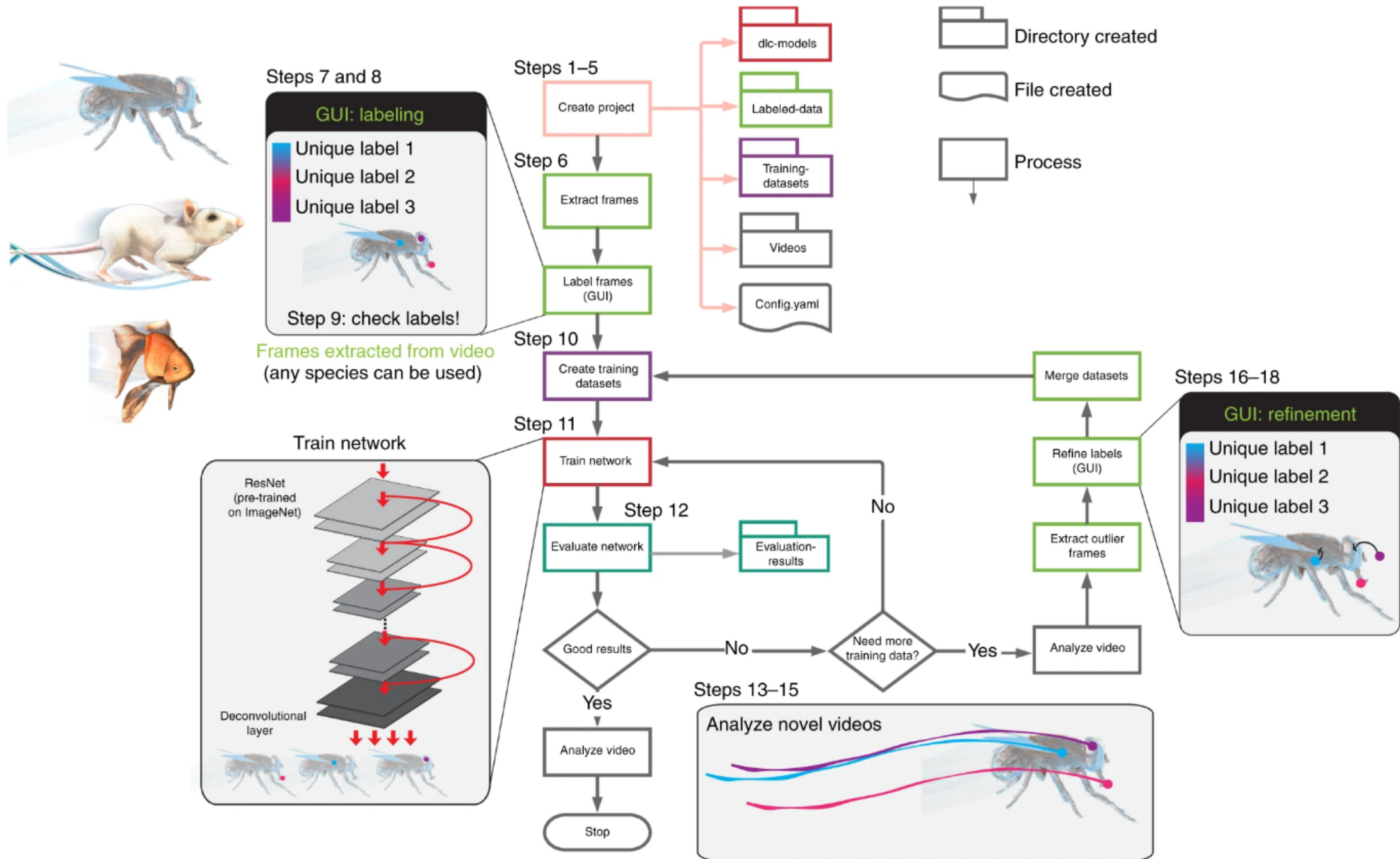
Using DeepLabCut for 3D markerless pose estimation across species and behaviors

[Tanmay Nath](#), [Alexander Mathis](#), [An Chi Chen](#), [Amir Patel](#), [Matthias Bethge](#) & [Mackenzie Weygandt Mathis](#) 

- https://deeplabcut.github.io/DeepLabCut/docs/standardDeepLabCut_UserGuide.html
- <https://sirocampus-86d7ce.pages.gitlab.lrz.de/tutorials/deeplabcut/> by Arash Shahidi

DLC for different species





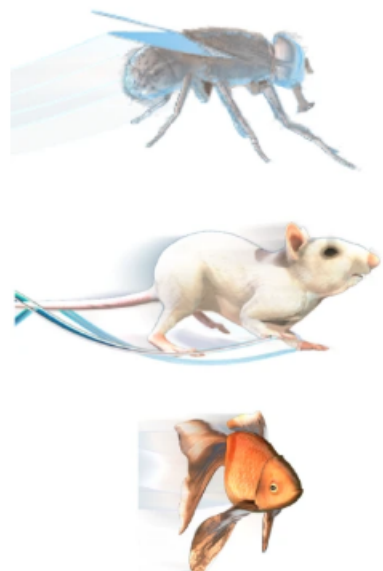
Requirement

GPU; we recommend using a GPU with at least 8 GB of memory, such as the NVIDIA GeForce 1080 or 2080. Alternatively, a CPU is sufficient, but training/evaluation of the network steps is considerably slower²⁷. GPUs with less memory might also work well. This toolbox can also be used on cloud computing services (such as Google Cloud/Amazon Web Services)

Camera:

- There are no a priori limitations in terms of lighting;
- color or grayscale images are acceptable
- >2 camera for 3D trajectory

Installation: `pip install deeplabcut`: details check <https://deeplabcut.github.io/DeepLabCut/docs/installation.html>



Steps 7 and 8

GUI: labeling

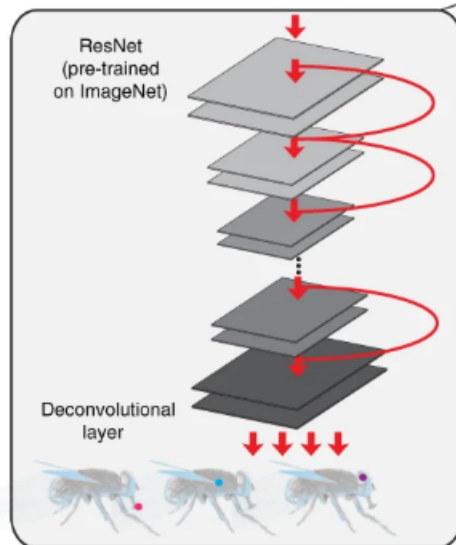
Unique label 1
Unique label 2
Unique label 3



Step 9: check labels!

Frames extracted from video
(any species can be used)

Train network



Steps 1–5

Create project

dlc-models

Labeled-data

Training-datasets

Videos

Config.yaml

Step 6

Extract frames

Label frames (GUI)

Step 10

Create training datasets

Step 11

Train network

Step 12

Evaluate network

Evaluation-results

Good results

Yes

Analyze video

Stop

Steps 13–15

Analyze novel videos



Directory created



File created



Process

Steps 16–18

GUI: refinement

Unique label 1
Unique label 2
Unique label 3



Merge datasets

Refine labels (GUI)

Extract outlier frames

Analyze video

No

No

Need more training data?

Yes

Create the project

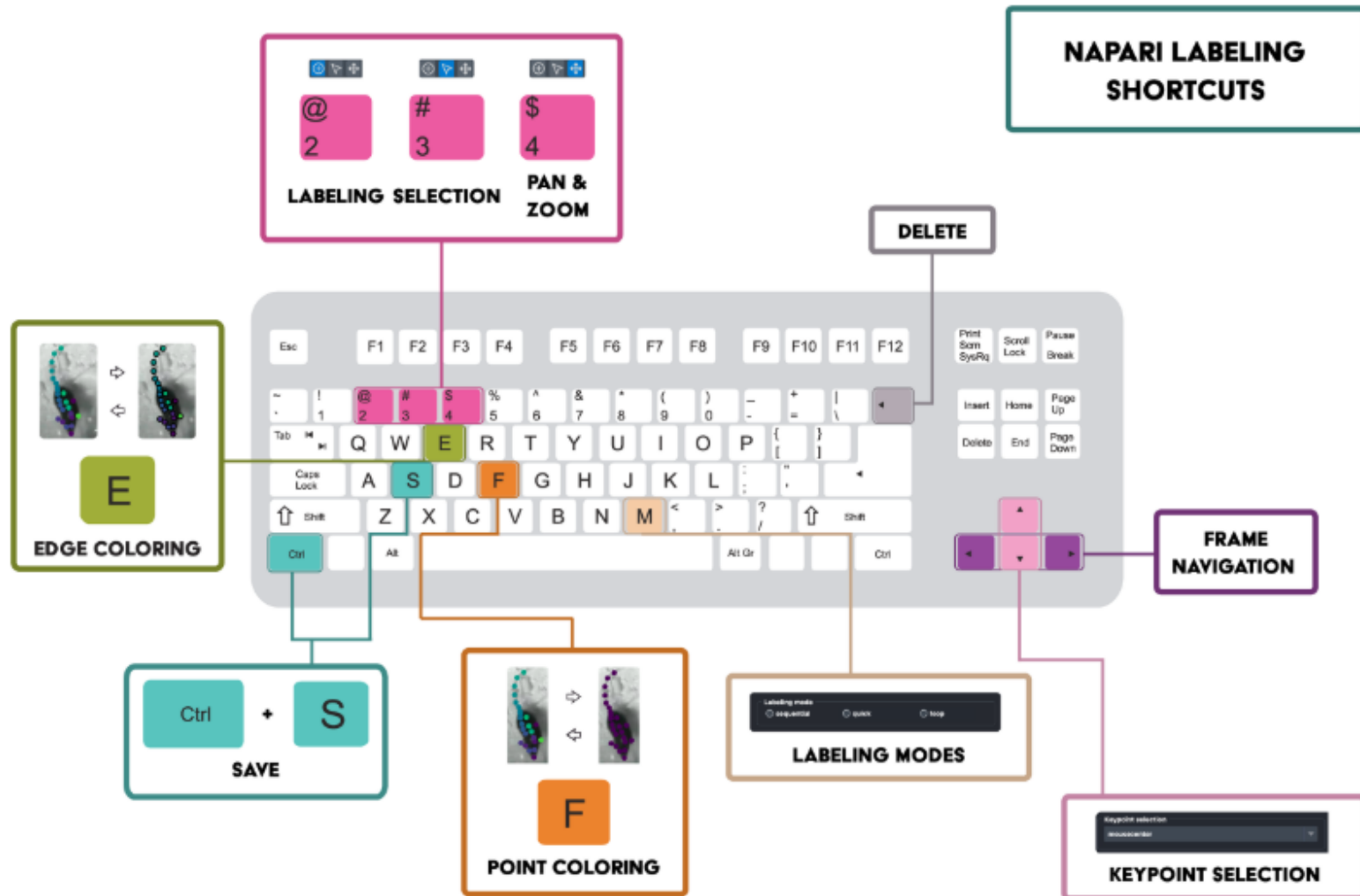
- *config.yaml*

```
deeplabcut.create_new_project(  
    "dlc_totutorial",  
    "miao",  
    [""], ["Full path of video2", "Full path of video3"],  
    working_directory="Full path of the working directory",  
    copy_videos=False,  
    multianimal=False  
)
```

Important parameters to edit after project creation

- **bodyparts**: List containing names of the points to be tracked. The default is set to bodypart1, bodypart2, bodypart3, objectA. Do not change after labeling frames (and saving labels). You can add additional labels later, if needed.
- **numframes2pick**: This is an integer that specifies the number of frames to be extracted from a video or a segment of video. The default is set to 20.
- **colormap**: This specifies the colormap used for plotting the labels in images or videos in many steps. Matplotlib colormaps are possible (https://matplotlib.org/examples/color/colormaps_reference.html).
- **dotsize**: Specifies the marker size when plotting the labels in images or videos. The default is set to 12.
- **alpha value**: Specifies the transparency of the plotted labels. The default is set to 0.5.

Label the data



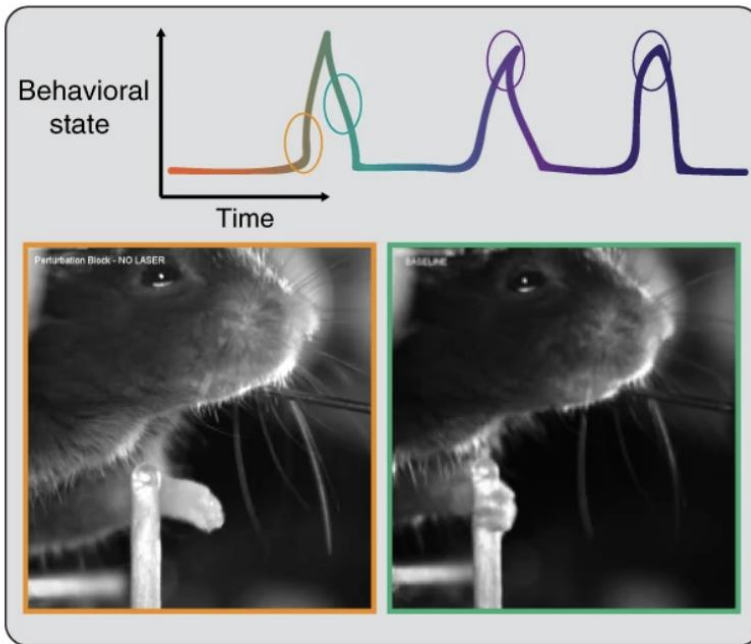
Select videos from which to grab frames:

Use videos with images from

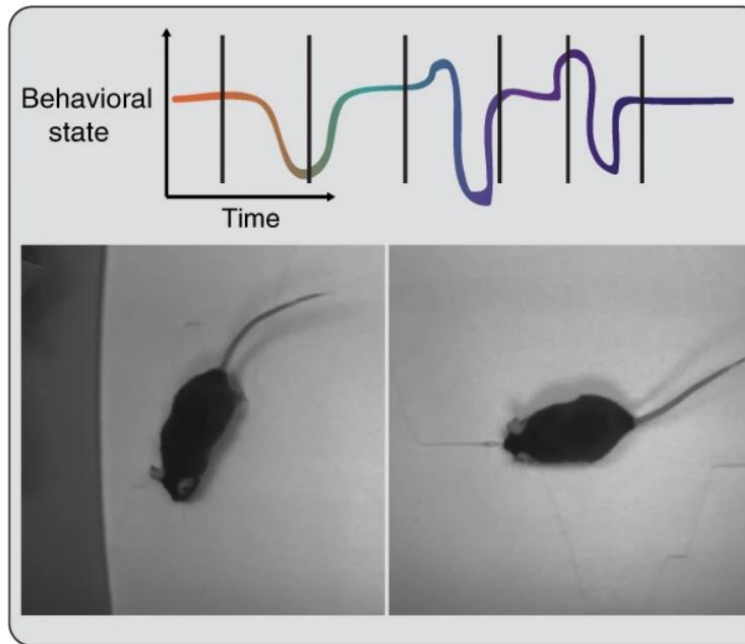
- Different sessions reflecting (if the case) varying light conditions, backgrounds, setups, and camera angles
- Different individuals, especially if they look different (i.e., brown and black mice)

3 methods for frame extraction to create a labeled train/test set

Image-based clustering (k -means)



Random temporal sampling (uniform)



GUI for manual frame grabbing



The toolbox contains three methods for extracting frames, namely, by clustering based on visual content, by randomly sampling in a uniform way across time, or by manually grabbing frames of interest using a custom GUI. The appropriate method should be selected depending on the studied behavior.

It is advisable to keep the frame size small

Extract frame if fail ...

```
deeplabcut.extract_frames(  
    config_path,  
    mode="automatic/manual",  
    algo="uniform/kmeans",  
    crop=True/False,  
    userfeedback=False  
)
```

Or:

`python code/dlc_extract_frames.py --config ./dlc-tutorial-miao-2026-03-09/config.yaml`

Labeling the marker

⚠ CRITICAL STEP

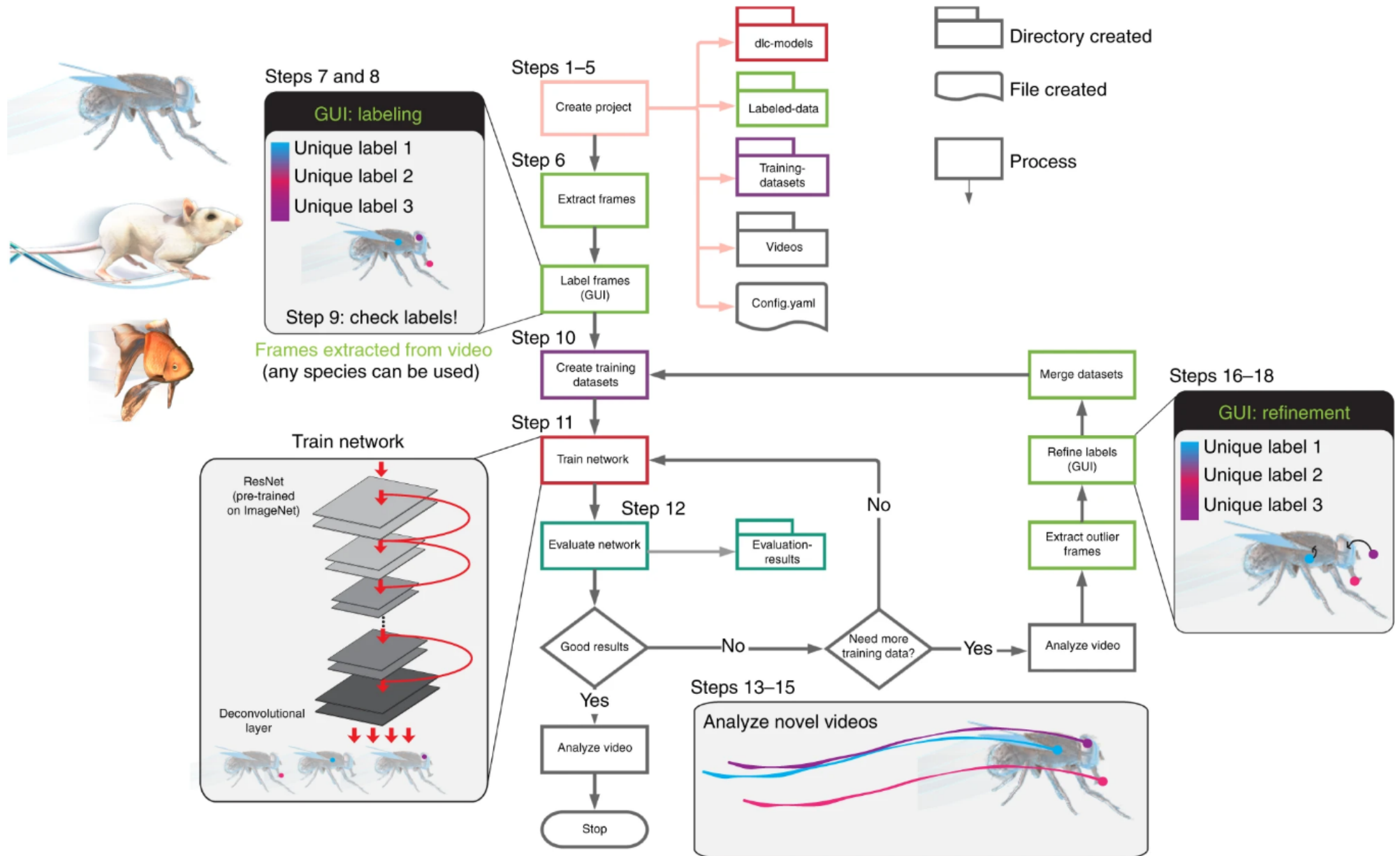
It is advisable to consistently label similar spots (e.g., on a wrist that is very large, try to label the same location). In general, invisible or occluded points should not be labeled. Simply skip the hidden part by not applying the label anywhere on the frame, or guess the location of body parts, in order to train a neural network that does that as well.

- Be consistent
- You can add or remove in the config file

- Nose, ears, spine, base of tail, tail tip

This allows the model to estimate:

- body orientation
- head direction
- tail motion



Train the network...

The set of arguments in the function starts training the network for the dataset created for one specific shuffle. Example parameters that one can call are given below:

```
train_network(config_path,shuffle=1,trainingsetindex=0,gputouse=None,max_snapshots_to_keep=5,
displayiters=1000,saveiters=20000, maxiters=200000)
```

Important parameters are as follows:

- `config_path`. Full path of the *config.yaml* file as a string (or a variable that points to the path).
- `shuffle`. Integer value specifying the shuffle index for the model to be trained. The default is set to 1.
- `trainingsetindex`. Integer specifying which training set fraction to use. By default it is the first index value listed (note that `Training Fraction` is a list in *config.yaml*).

Evaluation of network

```
deeplabcut.evaluate_network(config_path, plotting=True)
```

_locref — Location Refinement Map

- Shows the **offset vectors** that fine-tune the predicted keypoint location
- DLC first makes a coarse prediction on a grid, then **_locref** adds a small correction to get the **precise subpixel location**
- Visualized as arrows or a heatmap showing *which direction* the model is nudging each prediction
- Brighter/stronger signals = more confident directional refinement

_scmap — Score Map (Confidence Heatmap)

- Shows the **raw probability heatmap** for each body part
- Bright spots = where the model thinks the keypoint is located
- Each body part gets its own heatmap layer
- Ideally you want **tight, bright, focused blobs** on the correct location — diffuse or multiple blobs suggest the model is uncertain



Add videos

- `deeplabcut.analyze_videos(`
- `config_path,`
`["fullpath/analysis/project/videos/reachingvideo1.avi"],`
- `save_as_csv=True)`

Other tools

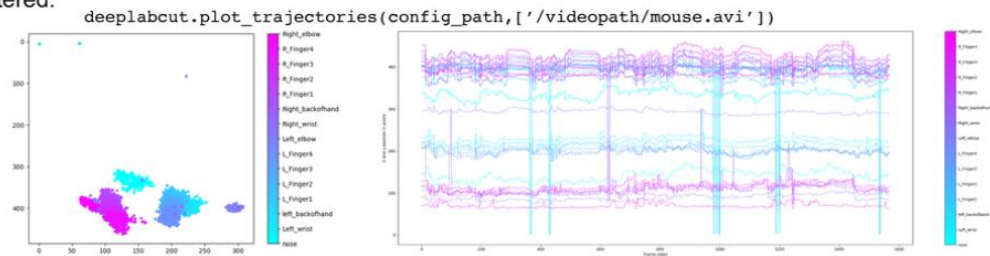
```
>> deeplabcut.filterpredictions(config_path,['video_path'],  
videotype='.avi')
```

```
>> deeplabcut.plot_trajectories(config_path,['Full path of  
video'])
```

```
>> deeplabcut.create_labeled_video(config_path,['Full path of  
video 1', 'Full path of video 2'])
```

```
deeplabcut.filterpredictions(config_path,['videopath/mouse.avi'], p_bound=0.1, ARdegree=5, MAdegree=1)
```

Unfiltered:



Filtered (ARIMA model):

